

PROMT PROPORCIONADO:

Actúa como asistente de programación para una práctica de Estructura de Datos en Python.

Contexto:

Ya tengo un proyecto desarrollado en clase con Python, programación orientada a objetos y una interfaz gráfica creada con Qt Designer. El proyecto ya permite convertir una expresión infija a posfija utilizando una clase Stack creada en clase.

Objetivo:

Necesito

complementar el proyecto para evaluar una expresión posfija utilizando la misma clase Stack. En esta práctica se usarán expresiones numéricas con operandos de un solo dígito, por ejemplo $2 + 3 * 4$, cuya expresión posfija sería $234*+$.

Instrucciones importantes:

1. No generes un proyecto nuevo desde cero.
2. No reemplaces la clase Stack existente.
3. No uses eval() de Python.
4. Respeta la organización actual del proyecto.
5. La solución debe estar hecha con programación orientada a objetos.
6. La interfaz gráfica debe seguir usando el diseño creado con Qt Designer.
7. Propón cambios mínimos para agregar la evaluación posfija.
8. Explica en qué archivo y en qué parte debo integrar el código.
9. Explica cómo se usa la pila durante la evaluación.
10. La evaluación

debe recorrer la expresión posfija, apilar los dígitos como números y aplicar las operaciones cuando encuentre operadores.

11. Proporciona pruebas para validar el funcionamiento.

Código existente:

```
[
from estructuras.lineales.nodoimport Node

class Stack(object):

    def __init__(self):
        self.top= None

# pregunta si la pila esta vacia o no
    def is_empty(self):
        return self.top is None

    def push(self,data):

        nuevo= Node(data)
        nuevo.next= self.top

        self.top= nuevo

    def pop(self):
        if self.top is None:
            return None

        valor= self.top.data
        self.top= self.top.next
```

```
#
```

Paso 4: Retornar el valor extraído (como dice la flecha verde: 'return valor')

```
return valor
```

```
def top_of_stack(self):
```

```
#
```

Verificar si la pila no tiene elementos

```
if self.top is None:
```

```
    return None
```

```
#
```

Retorna el valor del nodo en la cima pero SIN quitarlo de la pila

```
return self.top.data
```

```
def print_stack(self):
```

```
#
```

Verificar si está vacía

```
if self.top is None:
```

```
    return None
```

```
temp = self.top
```

```
print("---
```

Cima de la Pila ---")

```
while temp is not None:
```

```
    print(f"[{temp.data}
```

```
]")
```

```
    temp = temp.next
```

```
print("-----")
```

```
]
```

```
[fromestructuras.lineales.stackimport Stack
```

```
class ConvertidorExpresiones:
```

```
    def __init__(self):
```

```
        self.pila= Stack()
```

```
    def prioridad(self, operador):
```

```
        if operador == '$': #
```

```
Potencia
```

```
            return 3
```

```
        if operador in
```

```
        ('*', '/')
```

```
            return 2
```

```
        if operador in
```

```
        ('+', '-')
```

```
            return 1
```

```
        return 0
```

```
    def infija_a_posfija(self, expresion):
```

```
        salida= []
```

```
        for token in expresion:
```

```
            if token.isalnum():
```

```
                salida.append(token)
```

```
            elif token == '(':
```

```

self.pila.push(token)
    elif token == ')':

while not self.pila.is_empty() and self.pila.top_of_stack() != '(':

    salida.append(self.pila.pop())

self.pila.pop()
# eliminar '('
    else:

while (not self.pila.is_empty() and

        self.prioridad(self.pila.top_of_stack()) >= self.prioridad(token)):

    salida.append(self.pila.pop())

self.pila.push(token)

while not self.pila.is_empty():
    salida.append(self.pila.pop())

return " ".join(salida)

]

[from estructuras.lineales.stack import Stack

```

```

class ConvertidorExpresiones:

```

```

def __init__(self):
    self.pila= Stack()

def prioridad(self, operador):
    if operador == '$': #
Potencia
        return 3
    if operador in
('*', '/'):
        return 2
    if operador in
('+', '-'):
        return 1
    return 0

def infija_a_posfija(self, expresion):
    salida= []
    for token in expresion:
        if token.isalnum():

salida.append(token)
        elif token == '(':

self.pila.push(token)
        elif token == ')':

while not self.pila.is_empty() and self.pila.top_of_stack() != '(':

salida.append(self.pila.pop())

```

```

self.pila.pop()
# eliminar '('
    else:

while (notself.pila.is_empty())and

    self.prioridad(self.pila.top_of_stack())>= self.prioridad(token)):

salida.append(self.pila.pop())

self.pila.push(token)

whilenot self.pila.is_empty():
    salida.append(self.pila.pop())

return"".join(salida)

```

```

[fromPyQt5.QtWidgetsimport QDialog
from PyQt5import uic
from estructuras.lineales.convercionimport ConvertidorExpresiones

```

```

class LoadInterfazPila(QDialog):
    def __init__(self):
        super().__init__()
        self.ui= uic.loadUi('ui/untitled.ui',self)

        self.convertidor= ConvertidorExpresiones()

```

```

#
Conectar botón de conversión

self.btn_convertir.clicked.connect(self.convertir_infija_a_posfija)

def convertir_infija_a_posfija(self):
    expresion_infija = self.txt_expresion.text()
# corregido
    if not expresion_infija.strip():
        self.lbl_resultado.setText("Ingrese
una expresión válida.")
        return

    try:
        resultado_posfija = self.convertidor.infija_a_posfija(expresion_infija)
        self.lbl_resultado.setText(resultado_posfija)
    except Exception as e:
        self.lbl_resultado.setText(f"Error:{str(e)}")

'''

from PyQt5.QtWidgets import QDialog
from PyQt5 import uic
from estructuras.lineales.nodo import Node
from estructuras.lineales.conversion import
ConvertidorExpresiones

```



```

class LoadInterfazPila(QDialog):
    def __init__(self):
        super().__init__()
        self.ui =
        uic.loadUi('ui/untitled.ui', self)

        # Ya no usamos
        LinkedList, solo la pila

        self.operaciones =
        Node([])

        self.convertidor =
        ConvertidorExpresiones()

        # Conectar botón de
        conversión

        self.btn_convertir.clicked.connect(self.convertir_infija_a_posfija)

    def convertir_infija_a_posfija(self):
        expresion_infija =
        self.txt_expreccion.text()

        if not
        expresion_infija.strip():

        self.lbl_resultado.setText("Ingrese una expresión válida.")

        return

        try:

        resultado_posfija = self.convertidor.infija_a_posfija(expresion_infija)

```

```

self.lbl_resultado.setText(resultado_posfija)

except Exception as e:

    self.lbl_resultado.setText(f"Error:
{str(e)}")
'''
]

```

Resultado esperado:

- Primero analiza la estructura del código existente.
- Después propón la clase o método para evaluar la expresión posfija numérica.
- Luego indica cómo conectar ese método con el botón de la interfaz gráfica.
- Finalmente proporciona pruebas para comprobar que funciona.
- No omitas la explicación, porque debo entender el código para defenderlo en clase.

RESPUESTA GENERADA POR LA IA:

```

from estructuras.lineales.stack import Stack

```

```

class ConvertidorExpresiones:

```

```

    def __init__(self):

```

```
self.pila = Stack()
```

```
def prioridad(self, operador):
```

```
    if operador == '$': # Potencia
```

```
        return 3
```

```
    if operador in ('*', '/')
```

```
        return 2
```

```
    if operador in ('+', '-'):
```

```
        return 1
```

```
    return 0
```

```
def infija_a_posfija(self, expresion):
```

```
    salida = []
```

```
    for token in expresion:
```

```
        # === MODIFICACIÓN: Ignorar espacios en blanco ===
```

```
        if token == ' ':
```

```
            continue
```

```
        # =====
```

```
        if token.isalnum():
```

```
            salida.append(token)
```

```
        elif token == '(':
```

```
            self.pila.push(token)
```

```
        elif token == ')':
```

```
            while not self.pila.is_empty() and self.pila.top_of_stack() != '(':
```

```
                salida.append(self.pila.pop())
```

```
            self.pila.pop() # eliminar '('
```

```
        else:
```

```
            while (not self.pila.is_empty() and
```

```
        self.prioridad(self.pila.top_of_stack()) >= self.prioridad(token)):
    salida.append(self.pila.pop())
    self.pila.push(token)

while not self.pila.is_empty():
    salida.append(self.pila.pop())

return " ".join(salida)
```

```
def evaluar_posfija(self, expresion_posfija):
    # 1. Aseguramos que la pila esté vacía antes de iniciar
    while not self.pila.is_empty():
        self.pila.pop()

    # 2. Separamos la expresión por espacios
    tokens = expresion_posfija.split()

    for token in tokens:
        if token.isdigit():
            # Si es un dígito, lo convertimos a entero y lo apilamos
            self.pila.push(int(token))
        else:
            # Es un operador. Extraemos los dos últimos elementos.
            derecho = self.pila.pop()
            izquierdo = self.pila.pop()

            # Realizamos la operación matemática sin usar eval()
            if token == '+':
                resultado = izquierdo + derecho
```

```
elif token == '-':  
    resultado = izquierdo - derecho  
elif token == '*':  
    resultado = izquierdo * derecho  
elif token == '/':  
    # Usamos división clásica  
    resultado = izquierdo / derecho  
elif token == '$':  
    resultado = izquierdo ** derecho  
else:  
    raise ValueError(f"Operador no soportado: {token}")  
  
# Apilamos el resultado parcial  
self.pila.push(resultado)
```

```
# 3. Al final, el único valor restante en la pila es el resultado  
return self.pila.pop()
```